

## RESEARCH ARTICLE

# Democratization of deep learning for segmenting cartilage from MRIs of human knees: Application to data from the osteoarthritis initiative

Borja Rodriguez-Vila<sup>1,2,3</sup>  | Vera Gonzalez-Hospital<sup>3</sup> | Enrique Puertas<sup>3,4</sup> | Juan-Jose Beunza<sup>3,5</sup> | David M. Pierce<sup>6,7</sup> 

<sup>1</sup>Department of Electronics, Universidad Rey Juan Carlos, Madrid, Spain

<sup>2</sup>Medical Image Analysis and Biometry Laboratory, Universidad Rey Juan Carlos, Madrid, Spain

<sup>3</sup>Asalud, School for Doctoral Studies and Research, Universidad Europea de Madrid, Madrid, Spain

<sup>4</sup>Department of Computer Science and Technology, School of Architecture, Engineering and Design, Universidad Europea de Madrid, Madrid, Spain

<sup>5</sup>Department of Medicine, School of Biomedical and Health Sciences, Universidad Europea de Madrid, Madrid, Spain

<sup>6</sup>Department of Mechanical Engineering, University of Connecticut, Storrs, Connecticut, USA

<sup>7</sup>Department of Biomedical Engineering, University of Connecticut, Storrs, Connecticut, USA

## Correspondence

Borja Rodriguez-Vila, Department of Electronics, Universidad Rey Juan Carlos, Calle Tulipan 28933, Mostoles, Madrid, Spain.  
Email: [borja.rodriguez.vila@urjc.es](mailto:borja.rodriguez.vila@urjc.es)

David M. Pierce, Department of Mechanical Engineering, University of Connecticut, 191 Auditorium Rd, Storrs, CT 06269, USA.  
Email: [david.pierce@uconn.edu](mailto:david.pierce@uconn.edu)

## Funding information

Division of Civil, Mechanical and Manufacturing Innovation, Grant/Award Numbers: 1662429, CAREER 1653358

## Abstract

In this study, we aimed to democratize access to convolutional neural networks (CNN) for segmenting cartilage volumes, generating state-of-the-art results for specialized, real-world applications in hospitals and research. Segmentation of cross-sectional and/or longitudinal magnetic resonance (MR) images of articular cartilage facilitates both clinical management of joint damage/disease and fundamental research. Manual delineation of such images is a time-consuming task susceptible to high intra- and interoperator variability and prone to errors. Thus, enabling reliable and efficient analyses of MRIs of cartilage requires automated segmentation of cartilage volumes. Two main limitations arise in the development of hospital- or population-specific deep learning (DL) models for image segmentation: specialized knowledge and specialized hardware. We present a relatively easy and accessible implementation of a DL model to automatically segment MRIs of human knees with state-of-the-art accuracy. In representative examples, we trained CNN models in 6–8 h and obtained results quantitatively comparable to state-of-the-art for every anatomical structure. We established and evaluated our methods using two publicly available MRI data sets originating from the Osteoarthritis Initiative, Stryker Imorphics, and Zuse Institute Berlin (ZIB), as representative test cases. We use Google Colab for editing and adapting the Python codes and selecting the runtime environment leveraging high-performance graphical processing units. We designed our solution for novice users to apply to any data set with relatively few adaptations requiring only basic programming skills. To facilitate the adoption of our methods, we provide a complete guideline for using our methods and software, as well as the software tools themselves. Clinical significance: We establish and detail methods that clinical personal can apply to create their own DL models without specialized knowledge of DL nor specialized hardware/infrastructure and obtain results comparable with the state-of-the-art to facilitate both clinical management of joint damage/disease and fundamental research.

## KEYWORDS

deep learning, democratization, knee cartilage, magnetic resonance image, osteoarthritis, segmentation

## 1 | INTRODUCTION

Osteoarthritis (OA) is a debilitating disease of synovial joints, for example, knees, characterized by the degeneration and loss of articular cartilage, and subsequent joint function. OA afflicts nearly 20% of the US population, and its prevalence is projected to increase by about 40% in the next 25 years.<sup>1</sup> X-ray and magnetic resonance imaging (MRI) facilitate the assessment of knee joint degeneration, including the progression of OA, with the aim of specifying treatments.

Segmentation of cross-sectional and/or longitudinal magnetic resonance (MR) images of articular cartilage facilitates both clinical management of joint damage/disease and fundamental research, for example, facilitating the estimation of the articular cartilage loss rate and other morphological changes.<sup>2</sup> Such quantitative measures allow clinical assessment of disease progression and are also an essential prerequisite for the creation and validation of computational models of human joints.<sup>3</sup> Manual delineation of these images is a time-consuming task susceptible to high intra- and interoperator variability,<sup>4</sup> and prone to errors. Thus, enabling reliable and efficient analyses of MRIs of cartilage requires automated segmentation of cartilage volumes.

Convolutional neural networks (CNNs) are a class of artificial neural networks inspired by biological processes that have revolutionized many tasks in computer vision,<sup>5</sup> including segmentation and classification of images. CNNs consist of multiple neurons connected on different layers; each connection associated to a weight. CNNs currently demonstrate the best accuracy, repeatability, and efficiency in the segmentation of cartilage from within medical images, with results superior to conventional methods.<sup>6,7</sup>

Norman et al.<sup>8</sup> presented one of the first deep CNNs to automatically delineate anatomical structures of the human knee from MRIs. This approach leveraged U-Net two-dimensional (2D),<sup>9</sup> a CNN explicitly oriented to image segmentation. Panfilov et al.<sup>10</sup> presented a similar network architecture but focused on robustness to heterogeneity in the data. Ambellanet et al.<sup>11</sup> and Gatti et al.<sup>12</sup> proposed connecting a 2D U-Net and a 3D U-Net to further increase robustness. Toward the same end, Kessler et al.<sup>13</sup> and Gaj et al.<sup>14</sup> implemented a different kind of CNN, a conditional generative adversarial network (cGAN).<sup>15</sup> Finally, Desai et al.<sup>16</sup> compared results from the International Workshop on OA Imaging Knee MRI Segmentation Challenge, where six research teams worked with the same data set to facilitate direct performance comparisons. One of the main conclusions from this effort was that completely different CNNs achieve similar performances independent of network architecture and training design (although some networks had marginally lower computing performance).

Despite recent advances and some excellent results, CNNs are not a solution to all potential difficulties. When trained on data sets of medical

images, often limited in size and diversity of clinical cases, CNNs generally incorporate data set bias and fail to generalize well to new domains. Network performance is highly dependent on the accuracy of the segmentations used as ground truth, especially when trained with a low number of volumes. In these conditions, the network also replicates the performance of the human delineator who created the training data. Moreover, training a network on a low number of high-quality images restricts the applicability of the network to highly controlled studies with homogeneous data, thus limiting applications in clinical settings where image quality may vary significantly.<sup>13</sup>

No artificial intelligence (AI) algorithm designed for research on imaging the human knee has been successfully translated into practical medical devices since the methods do not generalize across patient populations from different hospitals.<sup>17</sup> Hospitals interested in automating a specific delineation process generally cannot leverage an existing deep learning (DL) model trained with a public data set. Instead, these hospitals must develop and train their own DL models using their own data sets that represent their population of interest (including their treatment population imaged with their equipment and protocol).

Two main limitations arise in the development of hospital- or population-specific DL models for image segmentation: specialized knowledge and specialized hardware. First, DL is often intimidating for application novices. Recent initiatives promote generalized access to DL, arguing that the design and development of CNNs requires neither advanced math nor programming skills, but the barrier remains.<sup>18</sup> Second, DL demands significant resources and hardware, for example, graphical processing units (GPU) or remote supercomputers—which may not be readily available locally, to obtain results in acceptable timeframes.

We feel that the current state of DL allows the democratization of access to the design and implementation of DL-based solutions to specialized problems. In this study, we aim to democratize access to CNNs for segmenting cartilage volumes, generating state-of-the-art results for specialized, real-world applications in hospitals and research. We establish and detail methods that clinical personal (e.g., medical doctors or IT technicians) can apply to create their own DL solutions without specialized knowledge of DL nor specialized hardware/infrastructure, and obtain results comparable with the state-of-the-art. To these ends, we also provide our source codes and pretrained models to the public.

## 2 | MATERIALS AND METHODS

Following the conclusions of Desai et al.,<sup>16</sup> we provide the methods and codes to adapt U-Net 2D networks to establish specialized segmentation models. We use two publicly available MRI data sets with known ground

truths as representative examples to establish our methods and provide direct comparisons to results representing the current state-of-the-art. The first representative example model, trained using Imorphics data set (Section 2.1.1), segments femoral, tibial, and patellar cartilages, as well as both menisci. The second representative example model, trained using Zuse Institute Berlin (ZIB) data set (Section 2.1.2), segments both femoral and tibial bones and cartilages (only structures available in this data set). In the following sections, we describe the image data sets, the development tools, and the runtime environment. We also explain both the training and segmentation steps, and the methods for quantifying the performance of the representative models. We detail the whole process in Appendix 1.

## 2.1 | Image data sets

We established and evaluated the accuracy of our methods using two publicly available MRI data sets, Osteoarthritis Initiative (OAI) Imorphics and OAI ZIB, as representative test cases. The images used to create both data sets originated from the OAI, a longitudinal study of OA progression.<sup>19</sup> This multicenter project data used 3T Siemens MAGNETOM Trio scanners and a sagittal double echo steady state (DESS) sequence (MRI parameters: field of view = 14 cm, voxel size = 0.365 mm × 0.365 mm × 0.70 mm, 160 slices of 384 × 384 pixels).<sup>20</sup> We converted the MRIs and volumetric image masks from NPY (OAI Imorphics) and ITK MetaImage Header (MHD) (OAI ZIB) formats to a common 2D image format (PNG, although JPG or TIFF formats are also sufficient) before subsequent training.

### 2.1.1 | Imorphics data set

We used sagittal DESS images and the corresponding segmentation masks for femoral, tibial, patellar cartilage, and meniscus generated by Stryker Imorphics. This data set includes 88 subjects with OA between Kellgren–Lawrence (KL)<sup>21</sup> grade 1 and 4 scanned at baseline and 12 months, and curated from the baseline set publicly available within the OAI data set. We use the data set exactly as previously used for The International Workshop on Osteoarthritis Imaging Knee MRI Segmentation Challenge, that is, with the same arrangement, an approximate proportion of 70%-15%-15% for training, validating, and testing, respectively.<sup>16</sup> This arrangement results in 74 subjects for training and validating, and 14 for testing, which implies 128 and 28 volumes for training and testing, respectively. These two cohorts included approximately equal distributions of KL grade, body mass index (BMI), and sex.

### 2.1.2 | ZIB data set

We also used sagittal DESS images and corresponding segmentation masks for femoral and tibial cartilage generated by ZIB (Berlin, DE).<sup>11</sup> The ZIB data set includes 507 subjects with all degrees of OA (KL 0–4), with more cases having severe OA (KL ≥ 3) scanned at baseline and curated from the baseline set publicly available within the OAI

data set.<sup>19</sup> They delineated the femoral and tibial bones and cartilages, using a statistical shape model with manual adjustments performed by experts if necessary, and they provide 507 annotated 3D DESS volumes.<sup>22</sup> We randomly selected 300 subjects and divided these into two cohorts: 120 for training and validation and 180 for testing. We choose to use 120 volumes for training in order to train both models with a similar number of input subjects. Since the total data set was larger, we decided to use a larger number of subjects for testing. Again, the two cohorts included approximately equal distributions of KL grade, BMI, and sex.

## 2.2 | Development and runtime environment

The two main limitations for the development of DL models, as previously mentioned, are the knowledge barrier and specialized hardware. Here we establish (1) a software library to simplify the design, development, and use of specialized CNNs, and (2) a low-cost environment for accessing high-performance hardware accelerators to train and apply the resulting models.

### 2.2.1 | Google colab

We use Google Colab, or “Colaboratory” (Google Research), for editing and adapting the Python code available in the appendices (Appendices 2–4) and selecting the runtime environment that uses high-performance GPUs. Colab is a software tool that allows any user to write and run arbitrary Python code within a web browser. The user can modify the runtime environment at any time, for example, users can run their codes on their own local processing units (CPU or even GPU) or can select remote hardware accelerators such as high-performance GPUs. The tool also allows transparent communication with Google Drive, the file hosting service from Google that currently offers 15 GB free of charge. We leverage Google Drive for storing data throughout the process: the MRI and mask volumes before processing, the data files obtained during intermediate steps, and the final results.

Colab offers resources free of charge, but it includes dynamic, fluctuating usage limits, and does not provide guaranteed or unlimited resources (so it may time out during use). In the free version of Colab, notebooks can run for up to 12 h, depending on availability, but it is difficult to reach that limit. Colab Pro offers increased processing availability and time. Colab Pro+ offers background execution, which supports continuous code execution for up to 24 h.

### 2.2.2 | FastAI

We use *fastai* to facilitate access to specialized knowledge on DL. It is a free open-source library for DL with a broad community of users, thorough documentation, and with extensive tutorials.<sup>18,23</sup> Using this library, complex processes simplify to single lines of code using the associated functions, for example, loading and associating MR images

with their respective masks (*SegmentationLoader*), creating a U-Net (*unet\_learner*), tuning parameters to optimize results (*fine\_tune*), and even using the model to obtain a segmentation (*predict*).

## 2.3 | Training the CNN models

In Figure 1, we provide a general overview of our pipeline for training a CNN model. We outline the main steps here including: extracting the slices, loading the data, establishing the initial CNN, training the CNN, and saving the final CNN (for later use).

- **Extracting the slices.** Since the network we selected is designed for 2D images, we divide each MRI and its corresponding mask volumes from the train data set ( $N = 128$  for Imorphics and  $N = 120$  for ZIB) into its 160 sagittal slices. We save the 2D images from both the MRIs and masks as separate PNG files in two different folders, respectively, named *images* and *labels*. To simplify the following steps, we identified associated images from the MRI and corresponding mask with the same name, but the label image includes the suffix “\_P.” We provide the code for this step in Appendix 2.
- **Loading the data.** We use the folders established in the prior step to feed the Data Loader, a data structure used to train any DL model. We perform this step with a single line of code using the *fastai* library, where *directory* is the path to our folder in Google

Drive containing the *images* and *labels* subfolders.

```
dls = SegmentationDataLoaders.from_label_func(directory, bs = 8,
fnames = fnames, label_func = label_func, codes = codes) (see
Appendix 3).
```

- **Establish the initial CNN.** We create an initial version of our model by applying a pretrained U-Net on our Data loader. In *fastai*, we can use a generic model based on Resnet34, a deep CNN of 34 layers, and pretrained with a public database.

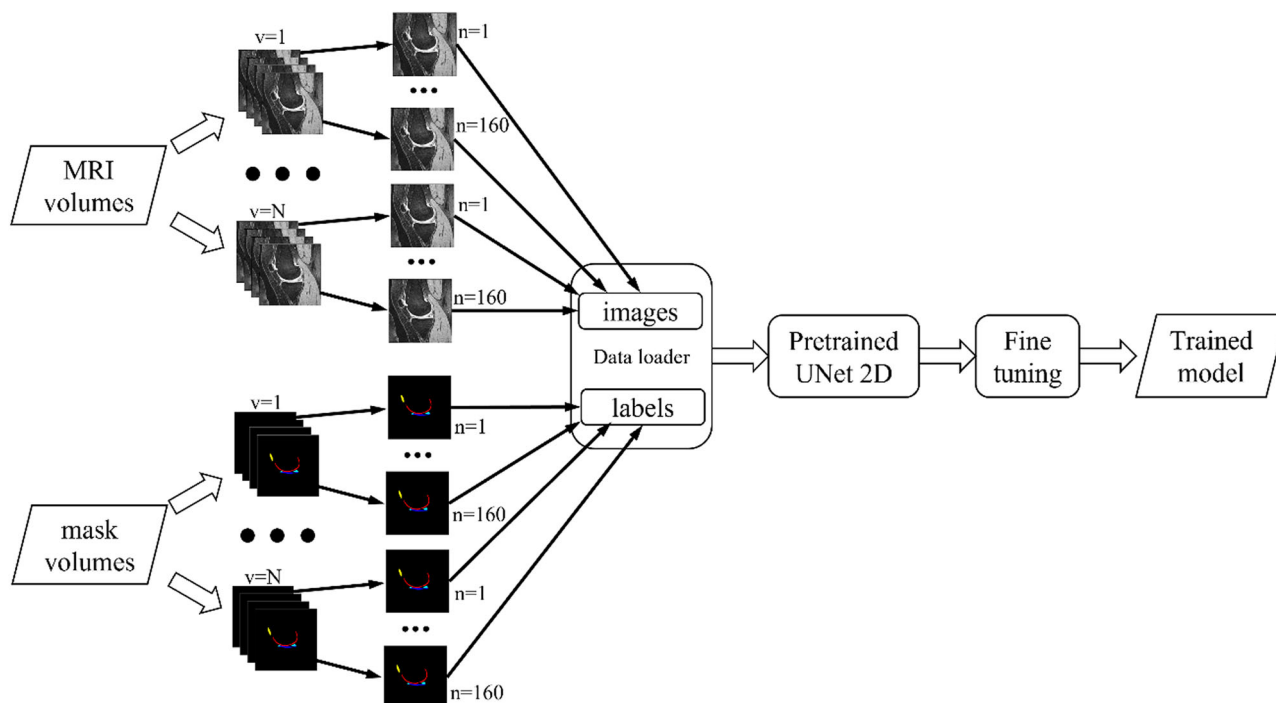
```
learn = unet_learner(dls, resnet34) (see Appendix 3).
```

- **Training the CNN.** During the training process, the network “learns” how to identify the different labels (image features) by automatically tuning its internal parameters. We must establish how many times we pass the entire data set through the neural network to update the weights of the neurons (i.e., epochs), this is an iterative process. In our case, six epochs (iterative trainings) obtain excellent results.

```
learn.fine_tune(6) (see Appendix 3).
```

- **Saving the final CNN.** We save the trained model for future use in segmentation.

```
learn.export('YourModel.pkl') (see Appendix 3).
```



**FIGURE 1** Pipeline for training a specialized convolutional neural networks model using U-Net two-dimensional and volumetric input data. Magnetic resonance imaging (MRI) and mask volumes consist of their separated slices. The “images” folder includes all of the saved MRI slices, while the “labels” folder includes all of the saved mask slices. Information from the data set feeds to the pretrained U-Net, which is then fine-tuned to produce the trained model.

## 2.4 | Using and evaluating the CNN models

In Figure 2, we provide a general overview of our pipeline for segmentation. The test cohorts (Sections 2.1.1 and 2.1.2) provide our gold standards to evaluate our final trained CNN models and we used them only once. Once we have a trained CNN model, we can very quickly segment an MRI volume. First, we must reorient the input volume to ensure correct extraction of the 2D sagittal slices. Here we use the LPS (left-right, posterior-anterior, superior-inferior) orientation. We then divide the MRI volume into its 160 sagittal slices, and we feed each of them to the trained model to perform a prediction (segmentation). We complete each of the 160 predictions using a single line of code.

`pred = learn.predict(stacked_img)` (see Appendix 4).

The result is a mask volume formed by the 160 2D predictions with no postprocessing.

To evaluate the performance of both trained CNN models, we employ measures that consider the volumetric overlap of predicted structures and the manual delineations, as well as the distances between their respective surfaces. We use the Sørensen–Dice Similarity Coefficient (DSC)<sup>24,25</sup> to evaluate the overlap between the manual delineations and our automated segmentations. DSC is a ratio between volumes, for example,  $X$  and  $Y$ , ranging from 0 (no overlap) to 1 (perfect overlap) as

$$DSC = 2 \frac{|X \cap Y|}{|X| + |Y|},$$

and reported as a fraction or a percent. We use the average symmetric surface distance (ASSD)<sup>26</sup> to evaluate the accuracy of our segmentations. ASSD is a boundary distance-based metric (here in mm) as

$$ASSD = \frac{\sum_{j=1}^{N_Y} D(X, y) + \sum_{j=1}^{N_X} D(Y, x)}{N_Y + N_X},$$

where  $Y$  is the automatic segmentation;  $X$  is the ground truth segmentation;  $D(X, y)$  is the distance of voxel  $y$  from surface  $Y$  to surface  $X$ ;

$D(Y, x)$  is the distance of voxel  $x$  from surface  $X$  to surface  $Y$ ; and  $N_Y$  and  $N_X$  are the number of pixels within each segmentation, respectively.

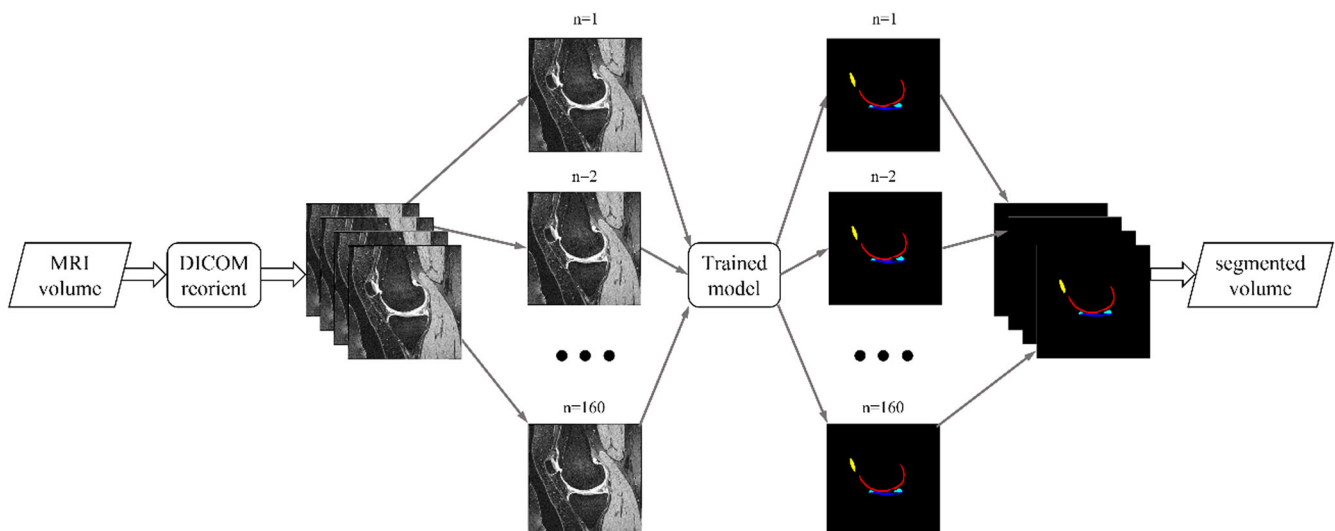
We perform two evaluations on both trained CNN models: first, the performance of each model against images from the data set used in training (the intended use), and second, the performance of each model against images from the other data set (not the intended use). To facilitate direct quantitative evaluations we compare our results with peer-reviewed, state-of-the-art results generated using the same data sets: namely Norman et al.,<sup>8</sup> Ambellan et al.,<sup>11</sup> Panfilov et al.,<sup>10</sup> Gaj et al.,<sup>14</sup> Gatti et al.,<sup>12</sup> Desai et al.,<sup>16</sup> for the Imorphics data set, and Abellan et al.<sup>11</sup> and Kessler et al.<sup>13</sup> for the ZIB data set. When evaluating the performance of each model with the “unitended” test data set (i.e., model trained with Imorphics evaluated with ZIB, or model trained with ZIB evaluated with Imorphics), we use only the tibial and femoral cartilage, since these are the only structures shared between data sets. We present the published DSC and ASSD values when reported. In published studies reporting results from the lateral and medial tibia cartilage and the lateral and medial menisci, we present averaged values.

To facilitate qualitative, visual comparisons, we also provide two representative comparisons. First, we provide representative 2D renderings of segmented structures using the trained models. Second, we provide representative 3D volumetric renderings of the ground truth and segmented cartilages compared for both the Imorphics and ZIB data sets and obtained using the Slicer 3D Model Maker module. Finally, we highlight the approximate average time required to complete the segmentations as the wall-clock time.

## 3 | RESULTS

### 3.1 | Training the CNN models

Training the CNNs took between 6 and 8 h in the Google Colab runtime environment using the available GPU hardware accelerators



**FIGURE 2** Pipeline for segmenting magnetic resonance images using the trained convolutional neural networks models



without the high-RAM option. Differences in training times relate to the different sizes of the training data sets and to the computational resources available in Google Colab at the specific time of training.

### 3.2 | Evaluating the CNN models

We obtain volumetric results on a new set of images in less than 8 min running a trained model within the Google Colab runtime environment and without using GPU hardware accelerators. In Figure 3, we provide representative 2D renderings of the segmented cartilages generated with our trained model overlaid (for comparison) on MRIs from both the Imorphics and ZIB data sets.

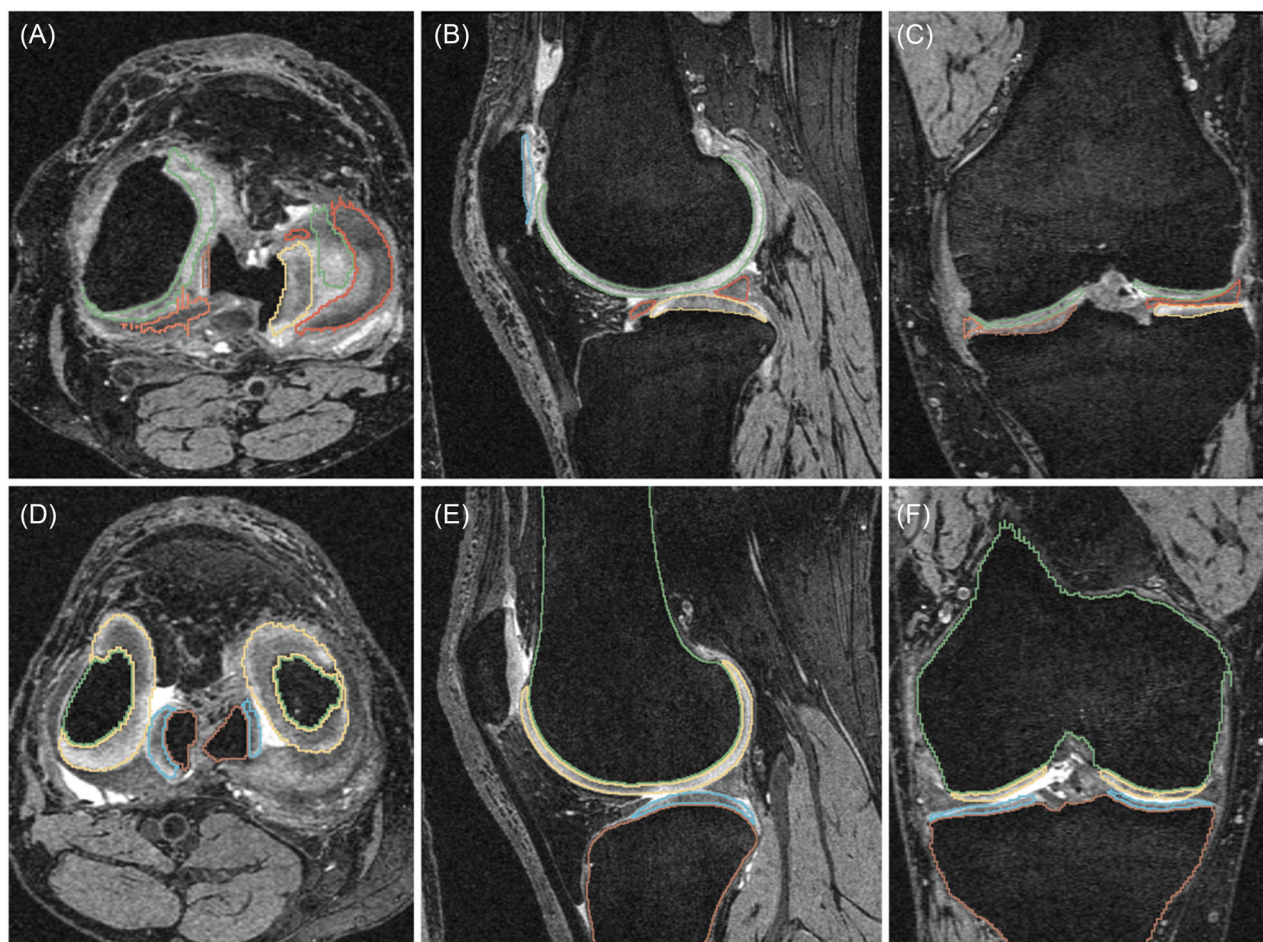
Our results show anatomical coherence for both trained CNN models, even though we perform the segmentation using only 2D sagittal slices (Figure 3B for Imorphics and Figure 3E for ZIB) with no prior volumetric information. Nonetheless, corresponding results in the transverse (Figure 3A for Imorphics and Figure 3D for ZIB) and coronal (Figure 3C for Imorphics and Figure 3F for ZIB) planes also show good results.

In Figure 4, we provide representative 3D volumetric renderings of the ground truth and segmented cartilages compared for both the Imorphics and the ZIB data sets.

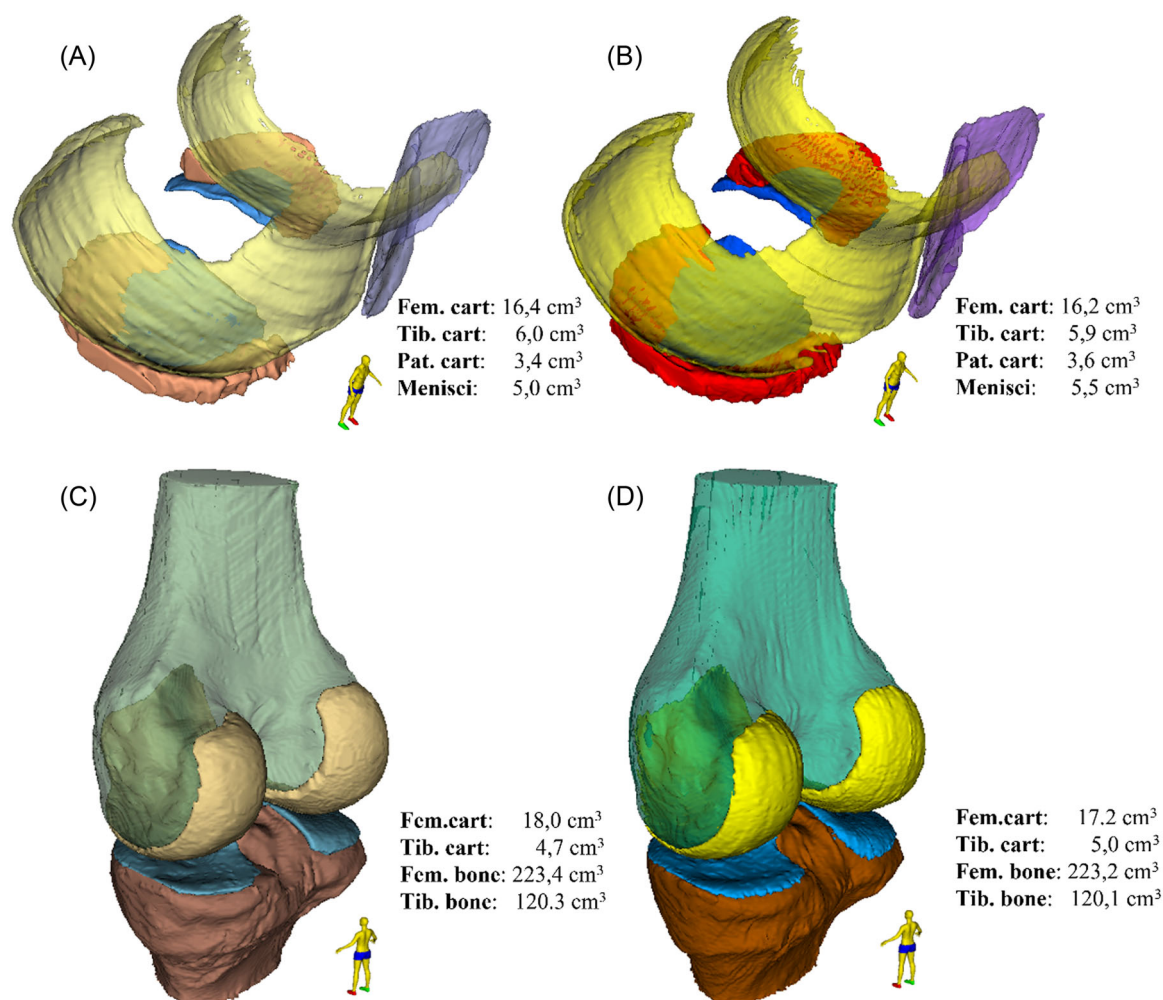
The 3D renderings of the segmentations (Figure 4) clearly show anatomical coherence for both trained CNN models.

In Table 1, we show the results of our CNN model trained and evaluated with the Imorphics data set and the comparison with corresponding models from the state-of-the-art trained with the same data set. In Table 2, we show the results of our CNN model trained and evaluated with the ZIB dataset and again the comparison with corresponding models from the state-of-the-art trained with the same data set. In Table 3, we show the results of our CNN model trained with the ZIB data set and evaluated with the Imorphics data set, and vice versa.

First, we detail the performance of each model against images from the data set used in training (the intended use). We obtain the best results for femoral and tibial bones within the ZIB data set, with a mean DSC > 98%. Femoral and tibial cartilages within the Imorphics data set also show good overlap values with the mean DSC



**FIGURE 3** Representative two-dimensional renderings of segmented structures using the trained models (in transverse [A, D], sagittal [B, E], and coronal [C, F] planes) overlaid on MRIs for both the Imorphics (A–C) and the ZIB (D–F) datasets. The Imorphics data set includes segmentations of menisci and femoral, tibial, and patellar cartilages, while the ZIB data set includes segmentations of femoral and tibial cartilages and bones.



**FIGURE 4** Representative three-dimensional volumetric renderings, including quantitative values, of ground truth (A, C) and segmented (B, D) volumes using the Imorphics (A, B) and the Zuse Institute Berlin (C, D) datasets.

approximately 0.90. Femoral and tibial cartilages within the ZIB data set show slightly lower values, but they are coherent and comparable with those from the literature. The mean DSC for menisci and patellar cartilage are above 87% and 84%, respectively (only available in the Imorphics data set). The ASSD are approximately 0.2 mm for all the structures in both data sets, demonstrating good correlation of the delineation boundaries in all cases.

Next, we detail the performance of each model against images from the other data set (not the intended use). In both cases performance of the models (trained with the Imorphics data set and evaluated using the ZIB test data set, and vice versa) leads to an important decrement in performance with the DSC approximately 80% for femoral cartilage and 70% for tibial cartilage (only possible comparisons).

## 4 | DISCUSSION

We present a relatively easy and accessible implementation of a DL model to perform automated segmentations of MR images of human knees with state-of-the-art accuracy. We obtained performance in

automated segmentations that rivals the state-of-the-art with minimum to no investment in hardware by leveraging free-access software tools and a basic personal computer. We designed our solution for users to apply to any data set with relatively few adaptations requiring only basic programming skills in Python.

### 4.1 | Training the CNN models

We trained CNN models to achieve state-of-the-art results within the proposed runtime environment (using powerful GPUs) in 6–8 h. There are two reasons behind this relatively fast training time: transfer learning and accelerated hardware. Transfer learning (a research area in machine learning) refers to storing knowledge gained while solving one problem and applying it to a different but related problem. One of the main applications is fine tuning a trained network by taking the parameters of a pretrained neural network (e.g., the initial U-Net available in fastai) and using this as the initialization of a new model trained on data from the same domain. In this way, we speed up training and overcome problems inherent to

**TABLE 1** Results of our CNN model trained and evaluated with the Imorphics data set and comparison with corresponding models from the state-of-the-art trained with the same data set

| Method                             | Femoral cartilage |             | Tibial cartilage |             | Menisci    |             | Patellar cartilage |             |
|------------------------------------|-------------------|-------------|------------------|-------------|------------|-------------|--------------------|-------------|
|                                    | DSC (%)           | ASSD (mm)   | DSC (%)          | ASSD (mm)   | DSC (%)    | ASSD (mm)   | DSC (%)            | ASSD (mm)   |
| Norman et al. <sup>8</sup>         | 86.7 ± 3.2        | NR          | 78.8 ± 3.3       | NR          | 77.1 ± 4.2 | NR          | 76.7 ± 9.1         | NR          |
| Ambellan et al. <sup>11</sup>      | 89.2 ± 2.4        | 0.19 ± 0.08 | 88.1 ± 3.8       | 0.21 ± 0.15 | NR         | NR          | NR                 | NR          |
| Panfilov et al. <sup>10</sup>      | 90.7 ± 1.9        | NR          | 89.7 ± 2.8       | NR          | 86.3 ± 3.4 | NR          | 87.1 ± 4.6         | NR          |
| Gaj et al. <sup>14</sup>           | 89.7 ± 2.3        | NR          | 88.9 ± 2.5       | NR          | 88.4 ± 3.4 | NR          | 84.2 ± 6.1         | NR          |
| Gatti et al. <sup>12</sup>         | 90.7 ± 2.2        | 0.17 ± 0.09 | 89.7 ± 2.6       | 0.19 ± 0.12 | NR         | NR          | 84.0 ± 12.8        | 0.35 ± 0.64 |
| Desai et al. <sup>16</sup> Team 1  | 88 ± 2            | 0.23 ± 0.04 | 87 ± 3           | 0.26 ± 0.14 | 84 ± 3     | 0.40 ± 0.09 | 83 ± 8             | 0.40 ± 0.46 |
| Desai et al. <sup>16</sup> Team 2  | 90 ± 2            | 0.19 ± 0.03 | 89 ± 3           | 0.25 ± 0.16 | 88 ± 3     | 0.31 ± 0.08 | 86 ± 7             | 0.23 ± 0.07 |
| Desai et al. <sup>16</sup> Team 3  | 90 ± 2            | 0.18 ± 0.03 | 89 ± 3           | 0.23 ± 0.14 | 88 ± 3     | 0.30 ± 0.07 | 85 ± 10            | 0.25 ± 0.14 |
| Desai et al. <sup>16</sup> Team 4  | 90 ± 2            | 0.18 ± 0.03 | 89 ± 4           | 0.25 ± 0.17 | 87 ± 3     | 0.32 ± 0.10 | 86 ± 7             | 0.23 ± 0.09 |
| Desai et al. <sup>16</sup> Team 5  | 87 ± 3            | 0.24 ± 0.06 | 85 ± 4           | 0.30 ± 0.18 | 83 ± 3     | 0.38 ± 0.08 | 81 ± 9             | 0.30 ± 0.12 |
| Desai et al. <sup>16</sup> Team 6  | 90 ± 2            | 0.19 ± 0.05 | 88 ± 3           | 0.29 ± 0.17 | 87 ± 4     | 0.38 ± 0.19 | 86 ± 7             | 0.34 ± 0.34 |
| Rodriguez-Vila et al. <sup>3</sup> | 89.9 ± 1.8        | 0.15 ± 0.02 | 88.1 ± 3.3       | 0.21 ± 0.11 | 87.1 ± 3.1 | 0.20 ± 0.10 | 84.7 ± 7.9         | 0.26 ± 0.08 |

Note: In the published studies evaluating more than one method, we present the best value for each category. In published studies reporting results from the lateral and medial tibia cartilage and the lateral and medial menisci we present averaged values.

Abbreviations: ASSD, average symmetric surface distance; CNN, convolutional neural networks; DSC, Dice Similarity Coefficient; NR, not reported.

**TABLE 2** Results of our CNN model trained and evaluated with the ZIB data set and comparison with corresponding models from the state of the art trained with the same data set

| Method                             | Femoral bone |             | Tibial bone |             | Femoral cartilage |             | Tibial cartilage |             |
|------------------------------------|--------------|-------------|-------------|-------------|-------------------|-------------|------------------|-------------|
|                                    | DSC (%)      | ASSD (mm)   | DSC (%)     | ASSD (mm)   | DSC (%)           | ASSD (mm)   | DSC (%)          | ASSD (mm)   |
| Abellan et al. 2019 <sup>11</sup>  | 98.6 ± 0.3   | 0.17 ± 0.05 | 98.5 ± 0.3  | 0.18 ± 0.06 | 89.9 ± 3.6        | 0.16 ± 0.07 | 85.6 ± 4.5       | 0.23 ± 0.12 |
| Kessler et al. 2020 <sup>13</sup>  | 98.5 ± 0.2   | 0.33 ± 0.12 | 98.5 ± 0.3  | 0.29 ± 0.07 | 89.7 ± 2.2        | NR          | 83.9 ± 4.0       | NR          |
| Rodriguez-Vila et al. <sup>3</sup> | 98.6 ± 0.3   | 0.19 ± 0.06 | 98.6 ± 0.4  | 0.19 ± 0.07 | 88.2 ± 2.6        | 0.22 ± 0.09 | 84.3 ± 4.5       | 0.24 ± 0.11 |

Note: In the published studies evaluating more than one method, we present the best value for each category. In published studies reporting results from the lateral and medial tibia cartilage and the lateral and medial menisci we present averaged values.

Abbreviations: ASSD, average symmetric surface distance; CNN, convolutional neural networks; DSC, Dice Similarity Coefficient; NR, not reported; ZIB, Zuse Institute Berlin.

**TABLE 3** Results of our CNN model trained with the ZIB data set and evaluated with the Imorphics data set, and vice versa

| Data set  |            | Femoral cartilage |             | Tibial cartilage |             |
|-----------|------------|-------------------|-------------|------------------|-------------|
| Training  | Evaluating | DSC (%)           | ASSD (mm)   | DSC (%)          | ASSD (mm)   |
| ZIB       | Imorphics  | 91.66 ± 1.51      | 0.18 ± 0.01 | 78.87 ± 3.41     | 0.35 ± 0.05 |
| Imorphics | ZIB        | 63.03 ± 16.44     | 0.81 ± 0.80 | 62.50 ± 11.74    | 0.63 ± 0.29 |

Abbreviations: ASSD, average symmetric surface distance; CNN, convolutional neural networks; DSC, Dice Similarity Coefficient; NR, not reported; ZIB, Zuse Institute Berlin.

small data sets. Second, we leveraged the remote computing resources available through Google Colab. The same process could take several days if performed on a common desktop or laptop CPU.

In training, importantly, we must establish how many times we pass the entire data set through the neural network to update the weights of the neurons (i.e., epochs), especially since this is an iterative process. A low number of epochs can result in poor results,

while too high a number can result in overfitting the model such that the trained CNN model does not generalize to new images. Based on our efforts here, we established that six epochs provide excellent results for this application (see Appendix 5 for more details).

Colab resources, including processing time, depend on current availability and fluctuate over time. Thus, the Colab runtime environment may time out while it is still training the model. Users



may need to be creative to find good times to train their models, even when using Colab Pro. Using Colab Pro, we have not had problems obtaining 6 h of processing time during periods of low demand (e.g., nighttime in the United States).

## 4.2 | Evaluating the CNN models

Both Tables 1 and 2 show the results of our segmentations, quantified with DSC and ASSD, obtained with two distinct data sets and compared against results from several current papers presenting state-of-the-art. These tables facilitate direct quantitative comparisons against previous studies leveraging the same two data sets. In Table 1, we show results from training and evaluating with the Imorphics data set. In this case, we obtained comparable results for both evaluation metrics and for every anatomical structure. No method generated better results for all categories. In Table 2, we show the corresponding results obtained with the CNN model trained and evaluated with the ZIB data set. Again the results are comparable for both evaluation metrics and no published model shows better results for all anatomical structures.

Absolute values of DSC and ASSD depend strongly on the segmented structure. Large and high-contrasted structures, like femoral and tibial bones, generally correspond to very high values of DSC. However, it is extremely difficult to obtain values of DSC > 98% with manual segmentations due to inter- and intrasubject variability. Contrary to DSC, ASSD considers only the contours of the segmentations and is highly dependent on the voxel size selected for the volumetric study. In our case, we obtain subvoxel accuracy, which implies that in a vast majority of the cases, the concordance between the contours we obtained, and the ground truth is below one voxel.

In Table 3, we also confirmed that in most cases, these models do not generalize well. Training a CNN model using the Imorphics data set and evaluating it using the ZIB test data set resulted in greatly reduced performance, with lower mean values and higher variability. Training a CNN model using the ZIB data set and evaluating it using the Imorphics data set, we obtained similarly poor results in the femoral cartilage, while showing a reduced overlap in tibial cartilage (mean DSC from ~90% to <80%, mean ASSD from 0.21 to 0.35 mm).

This reduction in performance remains even when both data sets use images from the same OAI database obtained using the same protocol. The file format of the data sets is different, and we did not study the influence of this difference on the resulting performance. Loss of performance may be even greater when applying trained CNN models to completely different data sets of images (e.g., from different patient cohorts and/or different imaging protocols).

Previously published ML models developed for segmentation of MRIs of the human knee (used here as references to state of the art) all incorporated highly specialized DL libraries (e.g., native TensorFlow,<sup>27</sup> PyTorch,<sup>28</sup> or Keras<sup>29</sup>) and trained using expensive, high-performance hardware. Conversely, our methods and resulting models use only simplified libraries and a low-cost runtime environment. Note that we performed each of the main steps to train and

evaluate the models (i.e., generating the data loader from the 2D images of the training data set, creating the initial U-Net model, fine tuning the model to optimize the results, exporting the trained model, and making predictions based on the trained model) using single lines of code thanks to the fastai functions. Additionally, Colab requires no configuration and currently gives free access to GPU computing where even better resources can be purchased if required. This is a low-cost alternative runtime environment for those with no access to specialized GPUs.

We obtained volumetric results on a new set of images in less than 8 min running a trained model within the Google Colab runtime environment and without using GPU hardware accelerators. Prediction times reported in the literature vary dramatically based on methods and hardware. Norman et al.<sup>8</sup> obtained results (volumetric segmentations) in approximately 8 s but did not report the computer hardware they employed. Panfilov et al.<sup>10</sup> developed their model from scratch using PyTorch and ran it on an Nvidia 2080 Ti GPU. Unfortunately, they do not reported training nor predicting times. Ambellan et al.<sup>11</sup> implemented the CNNs employing Keras using Theano<sup>30</sup> as the backend. Segmentation results took over nine minutes on a consumer-grade workstation with an Nvidia GeForce GTX 980 Ti GPU. Kessler et al.<sup>13</sup> implemented networks using PyTorch and performed all training on an Nvidia P6000 GPU card (3840 CUDA cores, 24 GB GDDR5X). Slice segmentations were obtained in 0.13 s, with volumetric results obtained in less than 1 min. Gaj et al.<sup>14</sup> used a framework of Keras and Tensorflow and trained it on the Owens Cluster of the Ohio Supercomputer Center with NVIDIA Tesla P100 GPUs. They also obtained volumetric segmentations in less than 1 min. Gatti et al.<sup>12</sup> performed experiments using a virtual machine with 12 CPU threads, 78 GB of RAM, and an NVIDIA Tesla V100 GPU on the Google Cloud Platform. With this approach, they obtained volumetric results in approximately 90 s.

Overall, our results confirm the conclusions of Desai et al.<sup>16</sup>: if using the same data set for training and testing, completely different CNNs obtain similar performances independent of network architecture and training design (although some networks have marginally lower computing performance). Thus, our approachable and economic implementation of a CNN offers comparable performance to those established by highly specialized research groups using highly specialized hardware, but it can be easily adapted by novices in DL and with limited access to computing resources. With our methods, almost anyone can create their own model based on their specific data set, that is, democratization of DL for segmenting cartilage from MRIs of human knees.

## 4.3 | Limitations and outlook

We aim to provide an accessible and economic approach to deep CNNs for nonexpert users, but we do not explain some of the concepts in full detail. Hence, we do not intend to teach the complete fundamentals of DL but to offer useful tools for creating powerful AI models oriented to novices.

The quantitative and qualitative results we obtained in this work relate to a specific clinical case, using MR images with specific dimensions obtained from a specific clinical database to segment specific anatomical structures. The adaptation of the proposed methods to different data sets should be straightforward and require only modifications to the code in Appendix 2 (to transform the volumetric images into 2D PNG images). However, adaptation of the code to a different clinical environment may require modifying the number of epochs and/or tuning the U-Net parameters provided in Appendix 5.

Due to the uniformity of the data sets, we did not use image augmentation to account for subtle differences in the orientation of the anatomy. Users may run out of RAM memory, but downsampling the images should solve this problem. The use of image formats that include compression (e.g., JPG with 8 bits instead of DICOM with 16 bits) is suboptimal, yet our CNN can still estimate the intensity and shape information and obtain segmentations at the state-of-the-art, all while reducing the use of RAM memory. With our approach, training times may vary significantly because Colab has dynamic usage limits that fluctuate with user load and they do not provide guaranteed or unlimited resources, in part because access to these resources is currently free of charge. This means that general usage limits, as well as idle timeout periods or available GPU types, vary over time. Users who require greater or more stable usage limits can use Colab Pro for a reasonable monthly fee (~\$10/month).

To facilitate adoption of our methods, we provide a complete guideline for using our methods and software solution, as well as the software tools themselves. In Appendix 1, we guide a user through the workflow and use of the tools. In Appendices 2–4, we offer complete, simple, and clean Python code that can directly be adapted and used by nonspecialized developers knowing only the basic concepts of programming in Python.

## AUTHOR CONTRIBUTIONS

**Borja Rodriguez-Vila:** Conceptualization, methodology, software, validation, writing original draft, review and editing. **Vera Gonzalez-Hospital:** Methodology, software, validation. **Enrique Puertas:** Methodology, review and editing. **Juan Jose Beunza:** Supervision, conceptualization, review and editing. **David M. Pierce:** Funding acquisition, conceptualization, resources, validation, writing original draft, review and editing.

## ACKNOWLEDGMENTS

NSF CAREER 1653358 and NSF 1662429.

## ORCID

Borja Rodriguez-Vila  <http://orcid.org/0000-0003-4779-0225>

David M. Pierce  <http://orcid.org/0000-0002-9713-1204>

## REFERENCES

- Lawrence RC, Felson DT, Helmick CG, et al. Estimates of the prevalence of arthritis and other rheumatic conditions in the United States. Part II. *Arthritis Rheum*. 2008;58:26-35.
- Ebrahimkhani S, Jaward MH, Cicuttini FM, Dharmaratne A, Wang Y, de Herrera AGS. A review on segmentation of knee articular cartilage: from conventional methods towards deep learning. *Artif Intell Med*. 2020;106:101851.
- Rodríguez-Vila B, Sánchez-González P, Oropesa I, Gomez EJ, Pierce DM. Automated hexahedral meshing of knee cartilage structures—application to data from the osteoarthritis initiative. *Comput Methods Biomech Biomed Engin*. 2017;20:1543-1553.
- Stammberger T, Eckstein F, Michaelis M, Englmeier KH, Reiser M. Interobserver reproducibility of quantitative cartilage measurements: comparison of B-spline snakes and manual segmentation. *Magn Reson Imaging*. 1999;17:1033-1042.
- Shen D, Wu G, Suk HI. Deep learning in medical image analysis. *Annu Rev Biomed Eng*. 2017;19:221-248.
- Heimann T, Morrison B, Styner MA, Niethammer M, Warfield SK. Segmentation of knee images: a grand challenge. MICCAI Workshop on Medical Image Analysis for the Clinic, Proceedings of the 13th International Conference Medical Image Computing and Computer-Assisted Intervention (MICCAI). Beijing, China. 2010.
- Renard F, Guedria S, Palma ND, Vuillerme N. Variability and reproducibility in deep learning for medical image segmentation. *Sci Rep*. 2020;10:13724.
- Norman B, Pedoia V, Majumdar S. Use of 2D U-Net convolutional neural networks for automated cartilage and meniscus segmentation of knee MR imaging data to determine relaxometry and morphometry. *Radiology*. 2018;288:177-185.
- Ronneberger O, Fischer P, Brox T. U-net: convolutional networks for biomedical image segmentation. In: Navab N, Hornegger J, Wells W, eds. *Medical Image Computing and Computer-Assisted Intervention—MICCAI*. Springer; 2015:234-241.
- Panfilov E, Tiulpin A, Klein S, Nieminen MT, Saarakkala S. Improving robustness of deep learning based knee MRI segmentation: mixup and adversarial domain adaptation. *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, 2019:450-459.
- Ambellan F, Tack A, Ehler M, Zachow S. Automated segmentation of knee bone and cartilage combining statistical shape knowledge and convolutional neural networks: data from the Osteoarthritis Initiative. *Med Image Anal*. 2019;52:109-118.
- Gatti AA, Maly MR. Automatic knee cartilage and bone segmentation using multi-stage convolutional neural networks: data from the osteoarthritis initiative. *MAGMA*. 2021;34:859-875.
- Kessler DA, MacKay JW, Crowe VA, et al. The optimisation of deep neural networks for segmenting multiple knee joint tissues from MRIs. *Comput Med Imaging Graph*. 2020;86:101793.
- Gaj S, Yang M, Nakamura K, Li X. Automated cartilage and meniscus segmentation of knee MRI with conditional generative adversarial networks. *Magn Reson Med*. 2020;84:437-449.
- Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets. *Proceedings of the International Conference on Neural Information Processing Systems*, 2014:2672-2680.
- Desai AD, Caliva F, Iriondo C, et al. The international workshop on osteoarthritis imaging knee MRI segmentation challenge: a multi-institute evaluation and analysis framework on a standardized dataset. *Radiol Artif Intell*. 2021;3:e200078.
- Si L, Zhong J, Huo J, et al. Deep learning in knee imaging: a systematic review utilizing a checklist for artificial intelligence in medical imaging (CLAIM). *Eur Radiol*. 2022;32:1353-1361.
- Howard J, Gugger S. Fastai: a layered API for deep learning. *Information*. 2020;11:108.
- Nevitt M, Felson D, Lester G. *The osteoarthritis initiative: protocol for the cohort study*. The Osteoarthritis Initiative, University of California; 2006.
- Williams TG, Holmes AP, Bowes M, et al. Measurement and visualisation of focal cartilage thickness change by MRI in a study

- of knee osteoarthritis using a novel image analysis tool. *Br J Radiol*. 2010;83:940-948.
21. Kellgren JH, Lawrence JS. Radiological assessment of osteoarthritis. *Ann Rheum Dis*. 1957;16:494-502.
  22. Seim H, Kainmueller D, Lamecker H, Bindernagel M, Malinowski J, Zachow S. Model-based auto-segmentation of knee bones and cartilage in MRI data. MICCAI Workshop on Medical Image Analysis for the Clinic, Proceedings of the 13th International Conference Medical Image Computing and Computer-Assisted Intervention (MICCAI). Beijing, China. 2010.
  23. Howard J. fastai. 2018. Available at <https://github.com/fastai/fastai>
  24. Dice LR. Measures of the amount of ecologic association between species. *Ecology*. 1945;26:297-302.
  25. Sorenson T. A method of establishing groups of equal amplitudes in plant sociology based on similarity of species content and its application to analyses of the vegetation on danish commons. kongelige danske videnskabernes selskab. *Biologiske Skrifter*. 1948;5: 1-34.
  26. Yeghiazaryan V, Voiculescu I. Family of boundary overlap metrics for the evaluation of medical image segmentation. *J Med Imaging*. 2018;5:1.
  27. Abadi M, Agarwal A, Barham P, et al. TensorFlow: large-scale machine learning on heterogeneous distributed systems. *arXiv*. 2016:1-19.
  28. Paszke A, Gross S, Massa F, et al. PyTorch: an imperative style, high-performance deep learning library. *Neural Information Processing Systems (NeurIPS)*, 2019;32:8024-8035.
  29. Chollet F. Keras. 2022. Available at <https://keras.io>
  30. The Theano Development Team. Theano: a Python framework for fast computation of mathematical expressions. *arXiv*. 2016:1-19.

**How to cite this article:** Rodriguez-Vila B, Gonzalez-Hospital V, Puertas E, Beunza J-J, Pierce DM. Democratization of deep learning for segmenting cartilage from MRIs of human knees: application to data from the osteoarthritis initiative. *J Orthop Res*. 2023;41:1754-1766. doi:10.1002/jor.25509

## APPENDIX 1: COMPLETE GUIDELINES

- (1) Select your own representative data set
  - (a) Load the DICOM images and save the volumes in MHD format, for example, using 3D Slicer or a similar program.
  - (b) Create your ground truth delineations and save them in MHD format, for example, using 3D Slicer or a similar program.
  - (c) Ensure that you have the same number of images as delineations, and that you use a coherent system for identifying both.
- (2) Divide the total data set into train (80%–90%) and test (10%–20%) data sets
  - (a) Randomly select between 10% and 20% of your patients
  - (b) Create a folder in Google Drive named *Test*
    - (i) Create two subfolders named *images* and *labels*
    - (ii) Store the MR image files of the selected patients into *Test/images*
    - (iii) Store the label image files of the selected patients into *Test/labels*

- (c) Create a folder in Google Drive named *Train3D*
  - (i) Create two subfolders named *images* and *labels*
  - (ii) Store the MR image files of the remaining patients into *Train3D/images*
  - (iii) Store the label image files of the remaining patients into *Train3D/labels*.
- (3) Divide the train 3D volumes into 2D images running code in Appendix 2.
  - (a) Create one folder named *TrainingDataset* with two subfolders, *images* and *labels*. After running Appendix 2 you will obtain:
    - (i) *images*: includes the 2D image files. File names consist of the patient number and the number of the slice.
    - (ii) *labels*: same file name as in *images*, but with the *\_P* suffix.
- (4) Fine tune the U-Net model using your own images
  - (a) Create a text file named *codes.txt* in the *TrainingDataset* folder. It should contain different names for each of the different structures you aim to segment. The order of the names should follow the order of the labels in the various mask files, and each name must be in a different line of the text file, for example, The *codes.txt* file for masks with 0 for the background, 1 for the femoral bone, 2 for the tibial bone, 3 for the femoral cartilage, and 4 for the tibial cartilage could be:
    - Background
    - FemBone
    - TibBone
    - FemCart
  - (b) *TibCartRuncode* in Appendix 3.
- (5) Predict your test segmentations running the code in Appendix 4 and your trained model.

## APPENDIX 2: CODE FOR CONVERTING FROM MHD VOLUMES TO PNG IMAGES (YOU CAN TEST THIS CODE USING THE ZIB DATA SET)

```
!pip install opencv-python
!pip install simpleitk
import cv2
import SimpleITK as sitk
import numpy as np
from pathlib import Path
from google.colab import drive
import os

# Connect to your Google Drive where your data is stored
drive.mount('/content/gdrive', force_remount=True)

# Directories for 3D images
MRI_dir = '/content/gdrive/MyDrive/Train3D/images/'
label_dir = '/content/gdrive/MyDrive/Train3D/labels/'

# Directories where 2D are going to be saved
save_MRI = '/content/gdrive/MyDrive/TrainingDataset/images/'
save_Label = '/content/gdrive/MyDrive/TrainingDataset/labels/'

# Create an array of filenames for both reference MR images and segmentations
```

```

MRI_files = []
label_files = []
for filename in os.listdir(MRI_dir):
    f = os.path.join(MRI_dir, filename)
    MRI_files.append(f)
for filename in os.listdir(label_dir):
    f = os.path.join(label_dir, filename)
    label_files.append(f)
MRI_files.sort()
label_files.sort()
# If you have correctly used the same number of MRI and label volumes
if len(MRI_files) == len(label_files):
    # For each patient in the folders
    for j in range(len(label_files)):
        # Only for MHD files
        if MRI_files[j].endswith('.mhd'):
            # Extract the number of the patient from the file name
            nameMRI = (os.path.splitext(MRI_files[j]))[0]
            # ZIB dataset uses the 7-digit OAI patient code to identify
            # the patient
            nameMRI = nameMRI[len(nameMRI)-7:len(nameMRI)]
            # Read MRI file
            MRI = sitk.ReadImage(MRI_files[j])
            # Orient image using the orientation of your choice, but be
            # consistent
            MRI = sitk.DICOMOrient(MRI, "LPI")
            # Read label file
            label3D = sitk.ReadImage(label_files[j])
            # Orient image using the orientation of your choice but be
            # consistent
            label3D = sitk.DICOMOrient(label3D, "LPI")
            for i in range(sitk.GetArrayFromImage(MRI).shape[2]):
                # for each one of the sagittal slices
                # zfill(3) represents the number always using 3 digits, e.g., 001
                nameMRISlice = nameMRI + '_' + str(i + 1).zfill(3) + '.png'
                nameLabelSlice = nameMRI + '_' + str(i + 1).zfill(3) + '_P.png'
                # Save labels
                label2D = sitk.GetArrayFromImage(label3D[i, :, :])
                save_labelPath = os.path.join(save_Label, nameLabelSlice)
                cv2.imwrite(save_labelPath, label2D.astype(np.uint8))
                # Save MRI slices
                slice2D = sitk.GetArrayFromImage(MRI[i, :, :])
                slice2D = np.where(slice2D < 256, slice2D, 255).astype
                    (np.uint8)
                imageMRI = (np.dstack((slice2D, slice2D, slice2D)))
                save_MRIPath = os.path.join(save_MRI, nameMRISlice)
                cv2.imwrite(save_MRIPath, imageMRI)

```

### APPENDIX 3: CODE FOR TRAINING

```

# Import needed libraries
!pip install -Uqqfastbook
!pip install --quiet --pre torchvision -f https://download.pytorch.org/whl/nightly/cpu/torch\_nightly.html

```

```

# Importing fastbook install fastai, but also other dependencies that
# are necessary
import fastbook
fastbook.setup_book()
from fastbook import *
import tensorflow as tf
from google.colab import drive
from pathlib import Path
# Connect to your Google Drive where your data is stored
drive.mount('/content/gdrive', force_remount=True)
# Directory
directory = Path('/content/gdrive/MyDrive/TrainingDataset')
# Codes
codes = np.loadtxt(directory/'codes.txt', dtype=str)
# Load the MRI 2D slices you have saved in TrainingDataset/images/
fnames = get_image_files(directory/"images")
# Create a function to automatically link images and labels
def label_func(fn): return directory/"labels"/f"{fn.stem}_P{fn.suffix}"
# Data Loader. Execute this line of code until the whole dataset is
# correctly loaded. It may take 2-3 times until all the images and labels
# are correctly loaded.
dls = SegmentationDataLoaders.from_label_func(directory, bs=8,
fnames = fnames, label_func = label_func, codes = codes)
# learn is our network. It is a U-Net with ResNet34 as encoder and
# initially pretrained using ImageNet
learn = unet_learner(dls, resnet34)
# Fine tune the model parameters using your dataset. Using ZIB
# dataset, 6 epochs were optimal.
learn.fine_tune(6)
# IMPORTANT: export.pkl format
learn.export('MyOwnModel.pkl')

```

### APPENDIX 4: CODE FOR PREDICTION OF ONE VOLUMETRIC STUDY

```

# Import necessary libraries
!pip install -Uqqfastbook
!pip install opencv-python
!pip install simpleitk
# Importing fastbook install fastai, but also other dependencies that
# are necessary
import fastbook
fastbook.setup_book()
from fastbook import *
import cv2
import SimpleITK as sitk
import numpy as np
import tensorflow as tf
import os
import time
from google.colab import drive
from pathlib import Path
# Connect to your Google Drive where your data is stored
drive.mount('/content/gdrive', force_remount=True)

```



```
# Redefine the function that automatically links images and labels
def label_func(fn): return directory+"/"+labels+"/"+fn.stem+"_P"+fn.suffix+"
# Load.pkl file with the trained model.
learn = load_learner('/content/gdrive/MyDrive/TrainingDataset/
MyOwnModel.pkl')
# select the MHD image that you want to segment
testFile = '/content/gdrive/MyDrive/Test/TestImage1.mhd'
if testFile.endswith('.mhd'):
    # If it is a MHD file, go on
    MRI = sitk.ReadImage(testFile)
    # Reorient image to ensure the code works properly. Be
consistent with the orientation used in Appendix 2.
    MRI = sitk.DICOMorient(MRI,"LPI")
    prediction = np.empty(sitk.GetArrayFromImage(MRI).shape)
    for i in range(sitk.GetArrayFromImage(MRI).shape[2]):
        # for each one of the 2D sagittal slices of the MRI volume
        # create a RGB image from the gray level image
        slice2D = sitk.GetArrayFromImage(MRI[i,:,:])
        stacked_img = (np.dstack((slice2D,slice2D,slice2D))).as-
type(np.uint8)
        # Call the learner and do the prediction(this is the core of the
method)
        pred = learn.predict(stacked_img)
        prediction[:, :, i] = np.array(pred[0])
# Create your own folder for the prediction and change the directory
to yours
save_dir = '/content/gdrive/MyDrive/Predictions/'
# This would be the name of your segmentation file
name = "TestImage1_pred.mha"
#Copy the header information from the MRI image and create 3D
image of the segmentation
image3D = sitk.GetImageFromArray(prediction.astype(np.uint8))
image3D.CopyInformation(MRI)
save_path = os.path.join(save_dir,name)
sitk.WriteImage(image3D, save_path)
```

## APPENDIX 5: SPECIFIC TERMINOLOGY AND ADVANCED SETTINGS

### (1) Data sets

- o Training Data set: The sample of data used to fit the model.

- o Validation Data set: The sample of data used to provide an unbiased evaluation of a model fit on the training data set while tuning model hyperparameters. The evaluation becomes more biased as skill on the validation data set is incorporated into the model configuration.
  - o Test Data set: The sample of data used to provide an unbiased evaluation of the final fit of the model on the training data set.
- (2) Loss function: The optimization process used to train neural networks requires a loss function to calculate the model error. Cross-entropy and mean-squared error are the two main types of loss functions used to train neural network models.
  - (3) Epoch: One forward pass and one backward pass of all the training examples. There is no default value for the number of epochs, and the user must select one in *learn.fine\_tune(6)*. Each epoch includes calculating values for the loss functions for both the Training and Evaluating data sets. The model is still learning while training loss and evaluating loss decrease in each epoch. When the train loss decreases but the validation loss stabilizes or decreases, the model is overfitting, that is, memorizing the images from the Training data set, so its performance with new images will suffer. In our case, the optimal value for the number of epochs is 6, but the user must select the value empirically.
  - (4) Batch size: The number of training examples in one forward/backward pass. The bigger the batch size, the greater the memory required. However, smaller batch sizes result in less accurate estimates of the gradient. In Fastai, the default value for the batch size is 64 but we use *bs = 8* to ensure that we do not run out of RAM memory.

*dls = SegmentationDataLoaders.from\_label\_func(directory, bs = 8, fnames = fnames, label\_func = label\_func, codes = codes)*

For the proposed U-Net model, we used the default values obtained using *dls = SegmentationDataLoaders.from\_label\_func* and *learn = unet\_learner(dls, resnet34)*.

We obtained the following values:

- Learning rate = 0.001
- Loss function = FlattenedLoss of CrossEntropyLoss()
- Activation function = ReLu
- Optimization function = Adam
- Momentum = [0.95,0.85,0.95]